

Assignment 3: A Simple CUDA Renderer

Author: simidaHK (independent study) **SUNet ID:** N/A **Test platform:** NVIDIA Tesla T4, 40 SMS, 14,918 MiB device memory, compute capability 7.5

Overview

This submission implements all three CUDA components of Assignment 3: SAXPY, a global-memory exclusive prefix sum with `find_repeats`, and an ordered, tile-based circle renderer. The renderer assigns one CUDA thread to each pixel, constructs an ordered list of circles intersecting each 16 x 16 image tile, and blends those circles into a thread-local pixel accumulator. This decomposition removes write conflicts by construction and preserves the required input order for transparent circles.

The eight graded renderer scenes passed the supplied CPU-reference image comparison at 1024 x 1024. The performance table below uses four-frame CUDA measurements collected on a Tesla T4. The precompiled x86 reference executable could not run in the development container because it required newer glibc, libstdc++, CUDA runtime, and FreeGLUT ABIs, so the reference column is taken from the assignment handout and the score is explicitly an estimate.

Part 1: CUDA SAXPY

Implementation

The CUDA SAXPY implementation allocates three device buffers, copies X, Y, and the initial result buffer from host to device, launches one CUDA thread per array element, copies the result back, and frees all device allocations. The kernel computes:

```
result[i] = alpha * X[i] + Y[i]
```

The grid size uses ceiling division, and the kernel checks `index < N`, so array lengths need not be multiples of the 512-thread block size.

Timing and performance interpretation

Two timing regions are reported. The overall region starts before host-to-device copies and ends after the device-to-host result copy. The kernel-only region starts immediately before the launch and calls `cudaDeviceSynchronize()` before stopping its timer. The synchronization is necessary because CUDA kernel launches are asynchronous with respect to the host.

The kernel-only result measures the high-bandwidth device-memory path: two float loads and one float store per output. It should substantially outperform a sequential CPU implementation for a sufficiently large array. The end-to-end measurement is much slower because a one-shot SAXPY performs little arithmetic per transferred byte and must move the arrays across the host-device interconnect. In this workload, transfer overhead can dominate the actual computation. This is consistent with the T4 device-memory bandwidth being far higher than the approximately 5.3 GB/s host-device bandwidth stated in the assignment.

Part 2: Exclusive Scan and Find Repeats

Exclusive scan

The implementation is an in-place CUDA version of the Blelloch exclusive scan from the assignment. The caller has already copied the input into the result buffer and allocated that buffer at the next power-of-two size.

The upsweep phase constructs a reduction tree. At level `two_d`, only $N / (2 * \text{two_d})$ logical threads are launched, and each active thread adds the left subtree total into the right subtree total. The last array element is then set to zero. The downsweep phase traverses the tree in reverse, swapping and accumulating partial sums to produce an exclusive prefix sum.

Successive kernel launches use the same default CUDA stream. Stream ordering therefore supplies the global synchronization required between scan levels; no device-wide synchronization is needed between every pair of kernels. The timing wrapper synchronizes once after the complete scan.

An important large-input bug was found during testing. The first version calculated `index * two_d_plus_1` before rejecting excess threads in a partially filled CUDA block. For a ten-million-element input rounded to 16,777,216, excess threads at the top tree levels overflowed a 32-bit integer, producing negative addresses and an illegal global-memory access. The final version first checks:

```
index < N / two_d_plus_1
```

and performs the multiplication only for active threads. A valid thread then always computes an address below `N`.

Find repeats

`find_repeats` uses three data-parallel stages:

1. A marking kernel writes 1 at index `i` when `input[i] == input[i + 1]`, otherwise 0.
2. The exclusive scan converts each mark into its compacted output position.
3. A scatter kernel writes each matching input index to `output[scan[i]]`.

The last mark is always zero, so `scan[length - 1]` equals the total number of repeats. This value is copied from device to host before the temporary scan buffer is freed. CUDA API return values are checked so allocation, launch, synchronization, and memory-access failures are reported at their source instead of becoming misleading output mismatches.

Part 3: Ordered CUDA Circle Renderer

Decomposition

The output image is divided into 16×16 pixel tiles. A CUDA block contains $16 \times 16 = 256$ threads and owns one tile; each thread exclusively owns one pixel in that tile. Circles are processed in batches of 256.

For each batch, thread `t` tests `circle batchStart + t` against the block's tile. A cheap conservative circle-box test rejects most nonintersecting circles before the exact circle-box test. Intersections produce a 256-element shared-memory flag array.

The supplied shared-memory exclusive scan converts flags into compacted offsets. Threads with a true flag store their original circle index into a shared `circleList`. Because exclusive scan is stable and batches are processed in ascending order, the candidate list preserves the original circle order.

Every pixel thread traverses the same ordered candidate list. It performs the final point-in-circle test at the pixel center, computes the scene-specific color and alpha, and blends into a local `float4` accumulator. The pixel is read from global memory once before all batches and written once after all batches.

Snow and ordinary scenes use template-specialized renderer kernels. The specialization moves the scene choice to the host-side kernel launch, allowing the compiler to eliminate the snow shading branch from the ordinary inner loop and vice versa.

Correctness: atomicity and order

Atomicity is satisfied without locks or atomic instructions. Each output pixel has exactly one owner thread, so no other thread can read or write that pixel during rendering. The complete read-blend-write sequence is therefore conflict-free.

Order is satisfied independently for every pixel. Batches advance from lower to higher circle indices, and the shared-memory compaction preserves order within a batch. Each pixel consequently blends contributing circles in exactly the input order used by the sequential reference renderer. No ordering is imposed between different pixels because the specification does not require it.

Threads outside a partial edge tile do not access image memory, but they still participate in every block barrier. This prevents deadlock when image dimensions are not exact multiples of the block dimensions.

Synchronization and communication

Each circle batch has three synchronization roles:

1. All flag writes complete before shared-memory scan begins.
2. Candidate-list writes and the final candidate count complete before pixel threads consume the list.
3. All pixel threads finish consuming a batch before the shared list is reused by the next batch.

The shared-memory scan contains its own block barriers. No cross-block synchronization is needed because blocks own disjoint pixels. Communication is reduced by tile-level circle rejection, a compact shared candidate list, and register-local pixel accumulation. The final design avoids global locks, global per-pixel candidate structures, and repeated global image writes.

Renderer results

Measurements use a 1024 x 1024 image and benchmark frames [0, 4). Student times are the reported per-frame render time. Correctness was checked separately against the CPU reference for every graded scene using seed 12345.

Scene	Handout ref (ms)	Student (ms)	T / Tref	Correct	Estimated score
rgb	0.2622	0.1574	0.60	Yes	9/9
rand10k	3.0658	3.8734	1.26	Yes	8/9
rand100k	29.6144	36.3300	1.23	Yes	8/9
pattern	0.4043	0.3215	0.80	Yes	9/9
snowsingle	19.7155	21.2294	1.08	Yes	9/9
biglittle	15.2422	16.7927	1.10	Yes	9/9
rand1M	230.4780	237.8824	1.03	Yes	9/9
micro2M	439.9369	447.8318	1.02	Yes	9/9
Total	-	-	-	8/8	70/72 estimated

The smallest scenes are dominated by launch, scan, and synchronization overhead, while the large-circle scenes benefit most from tile rejection and local accumulation. rand10k and rand100k remain slightly beyond the handout's full-credit threshold; the other six scenes are within 1.2 times the handout reference.

Development Process

The starter circle-parallel renderer demonstrates the core correctness failure: concurrent threads update the same four-component pixel without atomicity and in nondeterministic order. Locks could serialize individual pixel updates, but they would not naturally guarantee input order and would introduce severe contention.

The chosen solution instead changed ownership from circles to pixels. A naive pixel-parallel implementation would test every circle for every pixel, which is correct but wastes work. Tile-level culling amortizes the circle-box test across 256 pixels, and shared-memory scan produces a compact ordered list without global temporary storage. Local pixel accumulation then removes repeated framebuffer traffic.

Testing proceeded from small scenes to overlapping random and snow scenes, followed by all eight graded scenes. Prefix-sum testing similarly progressed from one million elements to larger inputs; the large tests exposed the integer-overflow bug that small tests could not trigger. Explicit CUDA error checking converted an implausibly fast incorrect result into the actionable `illegal memory access` diagnosis.

Reproduction

```
cd saxpy && make && ./cudaSaxpy
```

```
cd ../scan && make  
./checker.py scan  
./checker.py find_repeats
```

```
cd ../render && make  
./checker.py
```

For official score reproduction, the precompiled reference executables must be run in the course-specified ABI-compatible CUDA environment.